

PISTO: Proximal Inference for Stochastic Trajectory Optimization

Hongzhe Yu, Zinuo Chang, and Yongxin Chen

Abstract—Stochastic trajectory optimization methods like STOMP enable planning with non-differentiable costs, offering substantial flexibility over gradient-based approaches. We show that STOMP implicitly minimizes the KL divergence from a Boltzmann trajectory distribution, revealing an elegant Variational Inference (VI) structure underlying its updates. Building on this insight, we propose the *Proximal Inference for Stochastic Trajectory Optimization* (PISTO) algorithm that stabilizes the updates by augmenting the objective with a KL regularization between successive Gaussian proposals. This proximal formulation admits a trust-region interpretation and yields closed-form mean updates computable as expectations under a surrogate distribution. We estimate these expectations via importance-weighted Monte Carlo sampling, producing a simple, derivative-free algorithm that inherits STOMP’s ability to handle non-differentiable and discontinuous costs without modification. On robot arm motion planning benchmarks, PISTO achieves an 89% success rate—outperforming CHOMP (63%) and STOMP (68%)—while producing shorter, smoother paths at twice the speed of competing stochastic methods. We further validate PISTO on contact-rich MuJoCo locomotion and manipulation tasks, where it consistently outperforms both CEM and MPPI baselines in reward.

I. INTRODUCTION

Motion planning is central to robotics, enabling autonomous systems to generate collision-free, dynamically feasible trajectories in cluttered environments. While sampling-based methods such as PRM and RRT* [8, 16] provide strong exploration and asymptotic guarantees, their trajectories often require substantial post-processing for smoothness and dynamic feasibility. Consequently, trajectory optimization has become essential for high-performance planning in manipulation, mobile robotics, and aerial systems.

Optimization-based planners—including CHOMP [18, 28], sequential convex programming [20], and Gaussian process methods [11, 12]—represent trajectories in continuous time and optimize for smoothness, collision avoidance, and dynamic constraints. These approaches achieve impressive speed and scalability, but they rely on differentiable cost representations and remain susceptible to local minima in highly nonconvex environments.

Stochastic trajectory optimization broadens the class of tractable objectives by introducing sampling. STOMP [6] and path-integral methods such as PI² [21] perturb candidate trajectories and update them via cost-weighted averaging, enabling optimization under non-differentiable and discontinuous

costs. A natural question arises: what objective does STOMP implicitly optimize?

We show that STOMP minimizes the reverse KL divergence from a Boltzmann trajectory distribution, revealing an elegant variational inference structure underlying its updates. This connection places STOMP within a broader inference-based planning framework [3, 7, 22], where trajectory costs define a Boltzmann distribution whose high-probability mass concentrates on desirable plans. Gaussian variational inference formulations (GVIMP) [27] approximate this distribution with a structured Gaussian family, providing uncertainty quantification and a principled KL-divergence objective.

Building on this insight, we develop a proximal inference algorithm that stabilizes STOMP’s updates. Proximal methods [4, 13] augment each iterate with a penalty on the distance from the current solution, effectively creating a trust region that prevents overly aggressive steps. In the distributional setting, this naturally extends to KL-divergence penalties between successive proposals, controlling how rapidly the trajectory distribution evolves. By combining the variational inference perspective with proximal regularization, we obtain closed-form mean updates computable via importance-weighted Monte Carlo sampling—a simple, derivative-free algorithm that inherits STOMP’s flexibility while offering principled stability guarantees.

Contributions:

- We reveal that STOMP implicitly minimizes KL divergence from a Boltzmann trajectory distribution, establishing its variational inference structure and motivating principled algorithmic improvements.
- We propose *Proximal Inference for Stochastic Trajectory Optimization* (PISTO), which augments the reverse-KL objective with a proximal penalty between successive Gaussian proposals. This formulation admits a trust-region interpretation, stabilizes optimization dynamics, and yields closed-form mean updates computable via importance-weighted Monte Carlo sampling.
- We demonstrate PISTO’s effectiveness across diverse robotics tasks: on motion planning benchmarks, PISTO achieves 89% success rate—outperforming CHOMP (63%), STOMP (68%), and natural gradient descent (76%)—while running twice as fast as competing stochastic methods. On contact-rich MuJoCo tasks, PISTO consistently outperforms CEM and MPPI baselines in reward despite non-differentiable dynamics.

H. Yu and Y. Chen are with the School of Aerospace Engineering, Z. Chang is with the School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA; {hyu419, zchang40, yongchen}@gatech.edu

II. RELATED WORK

a) Planning as Inference: The interpretation of optimal control as probabilistic inference connects to linearly-solvable control [22], path-integral methods [7], and variational formulations in RL [10]. Within motion planning, Toussaint [24] applied message passing on graphical models, while Stein Variational approaches [17] maintain trajectory distributions via particle-based inference. Our work builds on the Planning as Inference paradigm [2, 5, 14, 15, 27], which showed that trajectory optimization with control-energy regularization is equivalent to KL-divergence minimization from a Boltzmann distribution. We solve the inference problem using a proximal inference algorithm that provides stable optimization and efficient Monte Carlo estimation. Other works under this paradigm are closely related. The variational and heteroscedasticity GP planners [5, 14] either rely on pre-defined kernels, induction point approximations, or start and goal states, which cannot be easily extended to control space optimization. Mixtures of Gaussian Processes for Trajectory Optimization (MG-PTO) [15] obtains a multi-modal trajectory using a STOMP-style cost-weighted stochastic gradient estimate. P-GVIMP [2] leverages GPU parallel computation to accelerate the mean and covariance updates in a gradient-descent landscape.

b) Trajectory Optimization: Gradient-based methods such as CHOMP [18, 28], TrajOpt [20], and GPMP [11] achieve fast convergence but require differentiable costs. Sampling-based methods relax this requirement: STOMP [6] uses cost-weighted averaging, path integral methods (PI² [21], MPPI [25]) derive updates from stochastic control, and CEM [19] fits distributions to elite samples. Our method differs by: (i) revealing STOMP's variational inference structure via KL-divergence; (ii) introducing proximal regularization for trust-region stability; and (iii) deriving closed-form updates amenable to importance sampling.

Our method introduces three key innovations over these approaches: (i) an explicit connection between STOMP's objective and KL-divergence minimization, revealing its variational inference structure; (ii) proximal regularization that stabilizes updates with a trust-region interpretation; and (iii) a reversed KL formulation yielding closed-form mean updates amenable to unbiased importance sampling.

III. STOCHASTIC TRAJECTORY OPTIMIZATION AS VARIATIONAL INFERENCE PLANNING

In this section, we establish the theoretical foundation of our approach by connecting classical stochastic trajectory optimization with variational inference. We begin by introducing the STOMP framework, followed by its interpretation as a variational inference problem.

A. Stochastic Trajectory Optimization (STOMP)

Stochastic Trajectory Optimization for Motion Planning (STOMP) [6] is a gradient-free optimization framework designed to handle non-differentiable and discontinuous cost functions. It explores the trajectory space by generating

stochastic perturbations around a nominal trajectory and updating it based on the exponentiated cost of these samples.

Let $Y \in \mathbb{R}^{n(T+1)}$ represent the mean of a T -length state trajectory, defined as $Y \triangleq \{x_t\}_{t=0}^T$. STOMP seeks to minimize the expectation of a cost functional over a proposal Gaussian distribution $\tilde{Y} \sim \mathcal{N}(Y, \Sigma)$:

$$\min_Y \mathcal{J}_1(Y) \triangleq \mathbb{E}_{\tilde{Y}} \left[S(\tilde{Y}) + \frac{1}{2} \tilde{Y}^\top R \tilde{Y} \right], \quad (1)$$

where $S(\tilde{Y}) = \sum_{t=0}^T V(\tilde{Y}_t)$ is the state-dependent potential (e.g., obstacle avoidance) and $R = A^\top A$ is the control cost matrix for double integrator dynamics, with $A \in \mathbb{R}^{(T-1) \times (T+1)}$ the finite-difference acceleration operator applying the stencil $[1, -2, 1]/\Delta t^2$ at interior nodes.

This stochastic formulation allows the planner to "smooth" the cost landscape, effectively escaping local minima that often trap purely gradient-based methods.

B. Variational Inference Interpretation of STOMP

The objective (1) admits a Variational Inference (VI) interpretation that recasts trajectory optimization as approximating an optimal path distribution. Any cost function over trajectories induces a Boltzmann distribution

$$p^*(\tilde{Y}) \propto \exp(-S(\tilde{Y})), \quad (2)$$

where low-cost trajectories receive high probability mass; this construction underlies the connection between optimal control and inference [7, 22] and path-integral methods [21]. In (1), the state-dependent potential $S(\tilde{Y})$ captures task-specific objectives such as obstacle avoidance, while the control cost $\frac{1}{2} \tilde{Y}^\top R \tilde{Y}$ with $R = A^\top A$ corresponds to a Gaussian smoothness prior $\mathcal{N}(0, R^{-1})$ favoring low-acceleration paths. Combining the two yields the main result connecting STOMP to VI.

Theorem 1 (Variational Inference Formulation of STOMP). *Consider the stochastic trajectory optimization objective (1). Let $\Sigma \succ 0$ be a fixed covariance and $R \succ 0$ be the control cost matrix. Minimizing $\mathcal{J}_1$ over the mean trajectory Y is equivalent to solving the variational inference problem:*

$$\min_Y D_{\text{KL}}(\mathcal{N}(Y, \Sigma) \parallel \mathbb{Y}^*), \quad (3)$$

where the target distribution is the energy-based posterior:

$$\mathbb{Y}^* \propto \exp(-S(\tilde{Y})) \mathcal{N}(0, R^{-1}). \quad (4)$$

Proof: See Appendix A-(a). ■

IV. PROXIMAL INFERENCE FOR STOCHASTIC TRAJECTORY OPTIMIZATION

We introduce a novel paradigm for solving the motion planning inference problem: Proximal Inference for Stochastic Trajectory Optimization (PISTO). PISTO reverses the argument order in the KL divergence and augments the objective with a Gaussian proximal term, converting each iteration into a proximal inference minimization whose optimizer admits a closed-form expectation representation amenable to Monte Carlo estimation.

A. Solution to the Reverse KL Minimization Problem

We first introduce an important moment-matching solution result in reverse KL minimization problems.

Lemma 1 (Reverse KL Minimization and Moment-matching Solution). *Consider the reverse KL objective obtained by swapping distributions in (3):*

$$\min_Y D_{\text{KL}}(\mathbb{Y}^* \parallel \mathcal{N}(Y, \Sigma)), \quad (5)$$

where $\mathbb{Y}^* \propto e^{-S(\tilde{Y})} \mathcal{N}(0, R^{-1})$. The gradient of the objective (5) with respect to Y is

$$\nabla_Y D_{\text{KL}}(\mathbb{Y}^* \parallel \mathcal{N}(Y, \Sigma)) = \mathbb{E}_{\mathbb{Y}^*} \left[-\Sigma^{-1}(\tilde{Y} - Y) \right], \quad (6)$$

and the optimal mean is given by

$$Y^* = \mathbb{E}_{\mathbb{Y}^*}[\tilde{Y}]. \quad (7)$$

With the choice $\Sigma = R^{-1}$ and proposal distribution $\tilde{Y} \sim \mathcal{N}(Y, R^{-1})$, the importance sampling estimator is

$$\hat{Y}^* = \sum_{m=1}^M \bar{w}_m(Y + \varepsilon_m), \quad \bar{w}_m = \frac{e^{-(S(Y + \varepsilon_m) + \varepsilon_m^\top R Y)}}{\sum_{j=1}^M e^{-(S(Y + \varepsilon_j) + \varepsilon_j^\top R Y)}}, \quad (8)$$

where $\varepsilon_m \sim \mathcal{N}(0, R^{-1})$ are i.i.d. samples.

The proof of Lemma 1 can be found in [26].

B. The main PISTO Formulation

Theorem 2 (Proximal Inference Update). *Consider the proximal update for the VI problem (3):*

$$Y_{k+1} = \arg \min_Y D_{\text{KL}}(\mathcal{N}(Y, \Sigma) \parallel \mathbb{Y}^*) + \frac{1}{\eta} D_{\text{KL}}(\mathcal{N}(Y, \Sigma) \parallel \mathcal{N}(Y_k, \Sigma)), \quad (9)$$

where $\eta > 0$ is the step size parameter, Y_k is the current iterate, and $\mathbb{Y}^* \propto e^{-S(\tilde{Y})} \mathcal{N}(0, R^{-1})$ is the target distribution. Then the proximal update is equivalent to KL projection onto a surrogate distribution:

$$Y_{k+1} = \arg \min_Y D_{\text{KL}}(\mathcal{N}(Y, \Sigma) \parallel \mathbb{Y}_k^*), \quad (11)$$

where the surrogate target $\mathbb{Y}_k^*$ admits the explicit form

$$\mathbb{Y}_k^* \propto \left(e^{-S(\tilde{Y})} \mathcal{N}(0, R^{-1}) \right)^{\frac{\eta}{\eta+1}} (\mathcal{N}(Y_k, \Sigma))^{\frac{1}{\eta+1}}. \quad (12)$$

Proof: Appendix A-(c). ■

Remark 1. The surrogate distribution $\mathbb{Y}_k^*$ geometrically interpolates between the true target $\mathbb{Y}^*$ and the current Gaussian approximation $\mathcal{N}(Y_k, \Sigma)$. As $\eta \rightarrow \infty$, the surrogate converges to the true target, recovering standard VI. For finite η , the proximal term regularizes the update, improving numerical stability and convergence.

C. Moment-matching Solution to the Proximal Inference

Eq. (11) indicates that each proximal iteration is itself a VI problem with respect to the surrogate distribution $\mathbb{Y}_k^*$, which is typically easier to handle than the original Boltzmann distribution. To compute the update, we solve the reverse KL-minimization projection

$$Y_{k+1} = \arg \min_Y D_{\text{KL}}(\mathbb{Y}_k^* \parallel \mathcal{N}(Y, \Sigma)). \quad (13)$$

To sample from $\mathbb{Y}_k^*$, we rewrite it in a tilted-Gaussian form by completing the square. Letting $\gamma \triangleq \frac{\eta}{\eta+1}$, we obtain

$$\log \mathbb{Y}_k^* \propto -\gamma S(\tilde{Y}) - \frac{1}{2}(\tilde{Y} - \mu_k)^\top P(\tilde{Y} - \mu_k), \quad (14)$$

where $P = \gamma R + (1 - \gamma)\Sigma^{-1}$ and $\mu_k = (1 - \gamma)P^{-1}\Sigma^{-1}Y_k$. This representation reveals that $\mathbb{Y}_k^*$ behaves like a Gaussian whose mean is shifted by the proximal term, modulated by an exponential tilt involving the cost function:

$$\mathbb{Y}_k^* \propto e^{-\gamma S(\tilde{Y})} \mathcal{N}(\tilde{Y}; \mu_k, P^{-1}). \quad (15)$$

Applying the gradient identity from (6) to the reverse KL-minimizing objective (13) and setting it to zero yields the following result.

Theorem 3 (Moment-Matching Update). *The solution to the reverse KL-minimization projection (13) satisfies the condition*

$$\nabla_Y D_{\text{KL}}(\mathbb{Y}_k^* \parallel \mathcal{N}(Y, \Sigma)) = \mathbb{E}_{\mathbb{Y}_k^*} \left[-\Sigma^{-1}(\tilde{Y} - Y) \right] = 0, \quad (16)$$

which admits the closed-form moment-matching update

$$Y_{k+1} = \mathbb{E}_{\mathbb{Y}_k^*}[\tilde{Y}]. \quad (17)$$

That is, the next iterate is simply the mean of the surrogate distribution $\mathbb{Y}_k^*$.

D. Importance Sampling

To estimate the expectation in (17), we employ importance sampling from a Gaussian proposal distribution. Similar to Eq. (8), the weight at iteration k is

$$w_k(\tilde{Y}) \triangleq \frac{e^{-\gamma S(\tilde{Y})} \mathcal{N}(\tilde{Y}; \mu_k, P^{-1})}{\mathcal{N}(\tilde{Y}; Y_k, P^{-1})} \propto e^{-\gamma S(\tilde{Y}) - \tilde{Y}^\top P(Y_k - \mu_k)}.$$

In the common choice $\Sigma = R^{-1}$, the expression simplifies considerably, as $P = R$ and $\mu_k = (1 - \gamma)Y_k$:

$$w_k(\tilde{Y}) \propto e^{-\gamma(S(\tilde{Y}) + \tilde{Y}^\top R Y_k)}. \quad (18)$$

Using these weights, we obtain a Monte-Carlo estimation of the proximal update:

$$Y_{k+1} \approx \sum_{m=1}^M \frac{e^{-\gamma(S(Y_k + \varepsilon_m) + \varepsilon_m^\top R Y_k)}}{\sum_{j=1}^M e^{-\gamma(S(Y_k + \varepsilon_j) + \varepsilon_j^\top R Y_k)}} (Y_k + \varepsilon_m), \quad (19)$$

where $\varepsilon_m \sim \mathcal{N}(0, R^{-1})$, $m = 1, \dots, M$ are sampled independently. This Monte Carlo estimator completes the proximal inference update, providing a stable and efficient mechanism for refining the Gaussian approximation at each iteration.

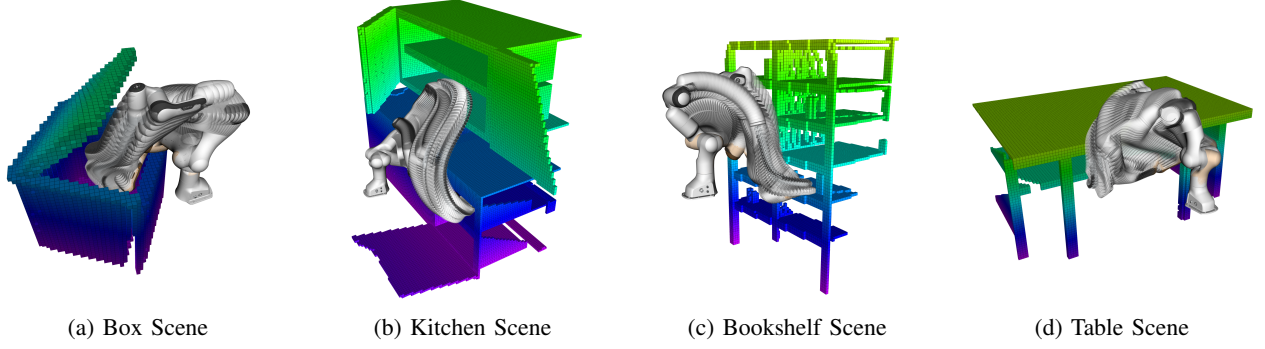


Fig. 1: Results of PISTO in different motion planning benchmarking scenes.

Algorithm 1 PISTO

Require: Accumulated cost $S(\cdot)$, smoothness matrix R , Cholesky factor L with $LL^\top = R^{-1}$, sample size M , proximal parameter η , current iterate Y_k , temperature τ

Ensure: Updated iterate Y_{k+1}

- 1: Compute $\gamma \leftarrow \frac{\eta}{\eta + 1}$
- 2: **Sample:** Draw $\{\varepsilon_m\}_{m=1}^M$ where $\varepsilon_m = Lz_m$, $z_m \sim \mathcal{N}(0, I)$
- 3: **Evaluate:** For each $m = 1, \dots, M$, compute importance weights

$$w_m \leftarrow \exp\left(-\frac{\gamma}{\tau} S(Y_k + \varepsilon_m) - (\varepsilon_m)^\top R Y_k\right)$$

- 4: **Normalize:** Compute normalized weights

$$\bar{w}_m \leftarrow \frac{w_m}{\sum_{j=1}^M w(j)}, \quad \forall m$$

- 5: **Update:** Compute weighted mean

$$Y_{k+1} \leftarrow Y_k + \sum_{m=1}^M \bar{w}_m \varepsilon_m$$

- 6: **return** Y_{k+1}
-

E. Policy Optimization for Contact-Rich Tasks

We now extend PISTO to policy optimization in the action space, where the decision variable is the *control sequence* $U = \{u_t\}_{t=0}^{T-1}$ and states evolve according to known dynamics models.

a) *Formulation:* Given initial state $x_0 \in \mathbb{R}^{d_x}$, consider the finite-horizon optimal control problem:

$$\min_U \sum_{t=0}^{T-1} c(x_t, u_t) \quad (20)$$

subject to the dynamics and control constraints:

$$x_{t+1} = g(x_t, u_t), \quad t = 0, \dots, T-1, \quad (21)$$

$$u_t \in \mathcal{U} \subseteq \mathbb{R}^{d_u}, \quad t = 0, \dots, T-1, \quad (22)$$

where $U = (u_0, \dots, u_{T-1}) \in \mathbb{R}^{T \times d_u}$, $c: \mathbb{R}^{d_x} \times \mathbb{R}^{d_u} \rightarrow \mathbb{R}$ is the stage cost, $g: \mathbb{R}^{d_x} \times \mathbb{R}^{d_u} \rightarrow \mathbb{R}^{d_x}$ is the dynamics model,

and $\mathcal{U} = [u, \bar{u}]$ is the admissible control set.

b) *Composite Structure of the Objective:* The state trajectory is uniquely determined by the initial condition and control sequence via recursive application of (21). We formalize this through the *flow map* $\phi_t: \mathbb{R}^{d_x} \times \mathbb{R}^{t \times d_u} \rightarrow \mathbb{R}^{d_x}$:

$$x_t = \phi_t(x_0, u_{0:t-1}) \triangleq \underbrace{g \circ g \circ \dots \circ g}_{t \text{ times}}(x_0, u_{0:t-1}), \quad (23)$$

with the convention $\phi_0(x_0) = x_0$. Substituting into (20) and defining the *rollout cost*

$$S(U; x_0) \triangleq \sum_{t=0}^{T-1} c(\phi_t(x_0, u_{0:t-1}), u_t), \quad (24)$$

the optimal control problem reduces to the form of (1):

$$\min_U \mathbb{E}_{\tilde{U}} \left[S(\tilde{U}; x_0) + \frac{1}{2} \tilde{U}^\top R \tilde{U} \right], \quad (25)$$

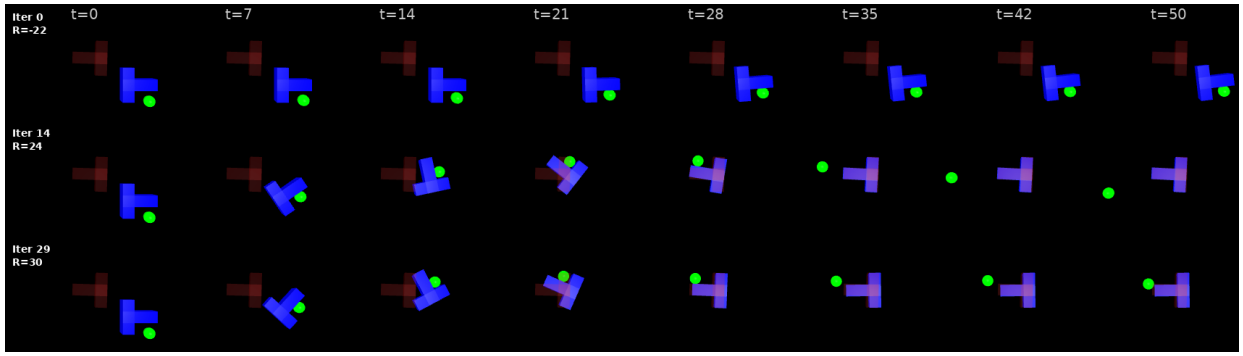
where the quadratic term $\frac{1}{2} \tilde{U}^\top R \tilde{U}$ regularizes the control sequence for temporal smoothness. Algorithm 1 applies directly in this setting, with each sample $\tilde{U}^{(i)} \sim \mathcal{N}(U, \Sigma)$ evaluated by rolling out the dynamics ϕ_t to obtain $S(\tilde{U}^{(i)}; x_0)$. This formulation enables PISTO to optimize directly in control space without requiring differentiability of g or c , and admits efficient parallelization of rollouts on modern GPU hardware.

F. Covariance and Proximal Step Size Annealing

To balance exploration and exploitation, we implement covariance scheduling with adaptive temperature scaling. We scale the smoothness matrix, $\bar{R} = \sigma_k \times R$, and use $\bar{R}$ matrix as the actual matrix that we sample from. The covariance scale σ_k follows cosine annealing:

$$\sigma_k = \sigma_{\text{final}} + \frac{1}{2}(\sigma_{\text{init}} - \sigma_{\text{final}}) \left(1 + \cos \left(\frac{\pi k}{K_{\text{max}}} \right) \right), \quad (26)$$

where k is the current iteration and K_{max} the maximum. We introduce an adaptive annealing scheme for the proximal step size parameter η governing the exploration-exploitation trade-off in importance-weighted trajectory optimization. We also introduce an additional temperature parameter τ to scale the impact of the adaptive proximal step size. The importance weights are $w_m \propto \exp(-\frac{\gamma}{\tau} S(Y_k + \varepsilon_m) - (\varepsilon_m)^\top R Y_k)$. We anneal η from small to large values via an exponential



(a) The PushT Task. Optimization time: 260.62(s).



(b) The Humanoid Running Task. Optimization time: 142.21(s).



(c) The Humanoid Standing Up Task. Optimization time: 168.11(s).

Fig. 2: The optimization results for contact-rich tasks.

schedule $\eta(t) = \eta_{\text{initial}} \cdot (\eta_{\text{final}}/\eta_{\text{initial}})^{t/T}$, where small η encourages exploration through nearly uniform weights while large η exploits high-reward samples with peaked weights. The temperature τ amplifies only the energy term, leaving regularization unaffected, enabling independent control over reward sensitivity and trajectory smoothness.

V. EXPERIMENTS

All experiments were run on a computer with an Intel Core i7-12800H CPU. The MuJoCo experiments were run on a computer with an NVIDIA RTX 4090 GPU. The code for this paper is implemented in C++ for the motion planning tasks, and in Python for the trajectory optimization tasks in MuJoCo.

⁰For STOMP, we employed the official implementation <https://github.com/ros-industrial/stomp>

A. Motion Planning for Robot Arms

For collision avoidance, we define the state cost $V(Y_t)$ using two formulations. The *signed-distance* cost approximates the robot as a union of spheres and evaluates

$$V_{\text{coll}}(Y_t) = \|h_{\delta}(d_{\text{sdf}}(F(Y_t)))\|_{\Sigma_{\text{obs}}}^2, \quad (27)$$

where $F(\cdot)$ is the forward kinematics, $d_{\text{sdf}}(\cdot)$ queries a pre-computed signed distance field (SDF), and the hinge function $h_{\delta}(\cdot)$ penalizes penetrations within margin δ . The non-differentiable *indicator* cost, used in MoveIt benchmarking experiments, imposes a fixed penalty on detected collision:

$$V_{\text{coll}}(Y_t) = W_{\text{obs}} \mathbf{1}_{\text{CollisionDetected}}(Y_t). \quad (28)$$

The weights Σ_{obs} and W_{obs} are task-dependent hyper-parameters.

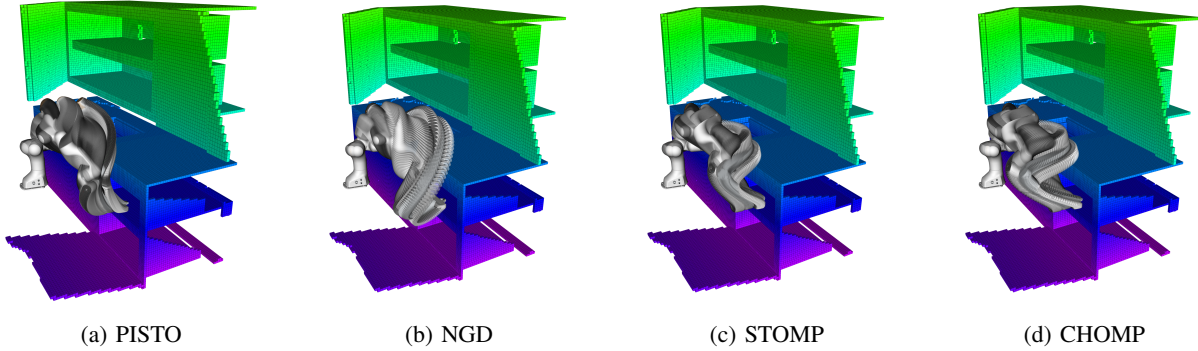


Fig. 3: Results of different planners in the Kitchen scene in the database.

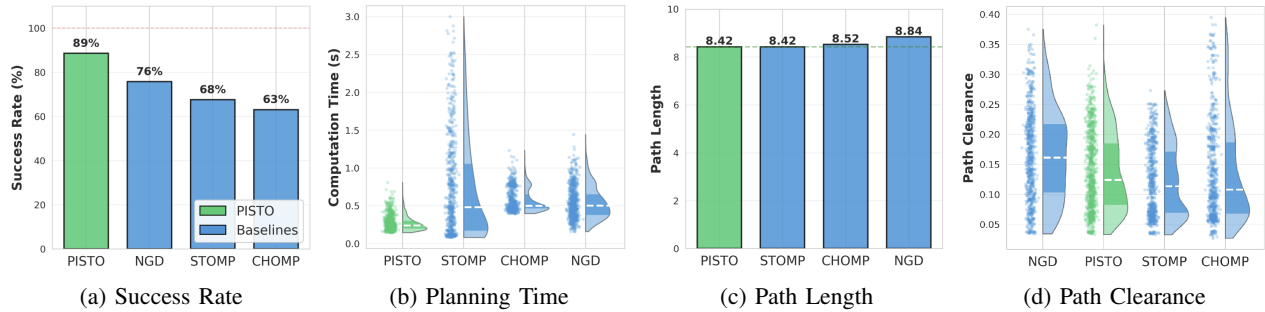


Fig. 4: Benchmark Performance Statistics

B. Policy Optimization Tasks

We tested the PISTO algorithm on various tasks defined in MuJoCo [23]. We used Bayesian optimization-based parameter sweeping to obtain the recommended parameters for each task. Figure 2 illustrates the optimization process for a standing up task for a 17-DOF humanoid robot. Table I records the achieved rewards for different tasks. *Planning and Runtime Results:* Table I records PISTO’s performance in contact-rich trajectory optimization tasks, compared with CEM and MPPI methods. The results are averaged over 50 independent runs. Our proposed PISTO algorithm significantly outperforms both baselines across all five contact-rich tasks, achieving approximately $1.5\times$ improvement on Walker2d and $2.8\times$ on HumanoidRun compared to the best baseline, while converting negative MPPI performance on PushT (-0.17) into a positive reward of 0.46 . PISTO also runs $1.4\times$ – $3.2\times$ faster than baselines on most tasks, with particularly notable speedups on HumanoidRun (36.5s vs. ~ 115 s). The tight standard deviations indicate reliable convergence despite discontinuous contact dynamics. Notably, PISTO achieves successful results for the 17-DOF humanoid running and standing-up tasks within minutes. These results validate PISTO’s robustness for trajectory optimization in hybrid dynamical systems.

C. Benchmarking with MoveIt Motion Planning Algorithms

We further benchmark PISTO against several representative optimization-based baselines using MoveIt, including the official implementations of STOMP and CHOMP, as well as a natural gradient descent (NGD) method [27]. Experiments are

conducted on the Franka Emika Panda across seven manipulation environments from the MotionBenchMaker [1] dataset, including *Kitchen*, *Bookshelf Tall*, *Bookshelf Thin*, *Table Pick*, *Table Under Pick*, *Box*, and *Cage*, with over 300 planning tasks in total.

For a fair comparison, all planners are initialized using the same joint-space straight-line interpolation between the start and goal configurations. We repeat each planning task 20 times with different random seeds. A run is considered successful if the resulting trajectory is collision-free and satisfies joint limits. Figure 4 summarizes the benchmarking results in terms of success rate, planning time, path length, and path clearance.

VI. CONCLUSION

We presented the Proximal Inference for Stochastic Trajectory Optimization (PISTO), a principled algorithm for motion planning formulated as Gaussian variational inference. By revealing STOMP’s implicit variational structure, we introduced a proximal formulation that regularizes updates via KL penalties, yielding closed-form moment-matching updates amenable to importance sampling. The resulting algorithm is simple, derivative-free, and parallelizable. Experiments demonstrated that PISTO achieves 89% success on motion planning benchmarks—outperforming CHOMP, STOMP, and natural gradient baselines—while additional MuJoCo experiments validated its effectiveness for high-dimensional, contact-rich tasks. Future work includes incorporating gradients when available, jointly optimizing mean and covariance, and extending to receding-horizon MPC.

TABLE I: Benchmarking Results: Motion Planning and Trajectory Optimization

Task	Method	Success/Reward		Time (s)	Path Length	Path Clearance
Motion Planning (7-DOF Manipulator)						
	CHOMP	63.05%		0.498	8.519	0.108
	STOMP	67.59%		<u>0.481</u>	<u>8.424</u>	0.113
	NGD	75.76%		0.499	8.838	0.161
	PISTO	88.57%		0.237	8.421	<u>0.124</u>
	Reward (Per step)			Runtime		
Task	PISTO	CEM	MPPI	PISTO	CEM	MPPI
Trajectory Optimization for Contact-Rich Tasks						
PushT	0.4559 ± 0.1291	0.1004 ± 0.1416	−0.1715 ± 0.1407	134.6235s ± 0.8061s	229.17s ± 1.48s	228.55s ± 1.44s
Hopper	1.2645 ± 0.0201	0.6931 ± 0.1390	0.9195 ± 0.1222	49.13s1.51s	58.76s ± 2.61s	58.53s ± 2.47s
Walker2d	1.2622 ± 0.0828	0.8603 ± 0.1107	0.7505 ± 0.1642	65.99s ± 1.50s	75.13s ± 2.57s	75.14s ± 2.72s
HumanoidRun	1.3385 ± 0.2921	0.4106 ± 0.2509	0.485 ± 0.3104	36.48s ± 1.64s	117.62s ± 14.42s	112.91s ± 12.98s
HumanoidStandUp	0.679 ± 0.058	0.4609 ± 0.008	0.529 ± 0.0390	65.29s ± 2.02s	55.18s ± 1.44s	54.24s ± 2.94s

APPENDIX

A. Proofs

a) *Proof of Theorem 1:* Define the accumulated state cost $S(\tilde{Y}) = \sum_{t=0}^T V(\tilde{Y}_t)$. Since the covariance Σ is fixed, the expectation of the quadratic deviation term simplifies via the trace identity $\mathbb{E}[(\tilde{Y} - Y)^\top R(\tilde{Y} - Y)] = \text{tr}(R\Sigma)$. Thus,

$$\mathcal{J}_1 = \mathbb{E}_{\tilde{Y}}[S(\tilde{Y})] + \frac{1}{2} \text{tr}(R\Sigma) + \frac{1}{2} Y^\top R Y.$$

We recognize that $\mathbb{E}_{\tilde{Y}}[S(\tilde{Y})] = -\mathbb{E}_{\tilde{Y}}[\log e^{-S(\tilde{Y})}]$. Combining this with the KL divergence between Gaussians,

$$\begin{aligned} D_{\text{KL}}(\mathcal{N}(Y, \Sigma) \parallel \mathcal{N}(0, R^{-1})) \\ = \frac{1}{2} (\text{tr}(R\Sigma) + Y^\top R Y - \log \det(\Sigma R) - (T+1)) \end{aligned}$$

where the last two terms are constants, we obtain

$$\mathcal{J}_1 = D_{\text{KL}}(\mathcal{N}(Y, \Sigma) \parallel \mathbb{Y}^*) + \text{const}, \quad (29)$$

by introducing the un-normalized $\mathbb{Y}^* \propto e^{-S(\tilde{Y})} \mathcal{N}(0, R^{-1})$. Here, the constant is independent of Y . Since only the mean Y is optimized, minimizing $\mathcal{J}_1$ is equivalent to minimizing the KL divergence to the target posterior $\mathbb{Y}^*$.

b) *Proof of Theorem 2:* Let q_Y and q_{Y_k} denote the density functions of $\mathcal{N}(Y, \Sigma)$ and $\mathcal{N}(Y_k, \Sigma)$, respectively. The proximal objective can be written as

$$\begin{aligned} \mathcal{J}_k &= D_{\text{KL}}(\mathcal{N}(Y, \Sigma) \parallel \mathbb{Y}^*) + \frac{1}{\eta} D_{\text{KL}}(\mathcal{N}(Y, \Sigma) \parallel \mathcal{N}(Y_k, \Sigma)) \\ &= \mathbb{E}_{\mathcal{N}(Y, \Sigma)} \left[\left(1 + \frac{1}{\eta} \right) \log q_Y - \log \mathbb{Y}^* - \frac{1}{\eta} \log q_{Y_k} \right]. \end{aligned}$$

Factoring out the coefficient $\frac{\eta+1}{\eta}$ yields

$$\begin{aligned} \mathcal{J}_k &= \frac{\eta+1}{\eta} \mathbb{E}_{\mathcal{N}(Y, \Sigma)} \left[\log q_Y - \log \left((\mathbb{Y}^*)^{\frac{\eta}{\eta+1}} (q_{Y_k})^{\frac{1}{\eta+1}} \right) \right] \\ &\propto D_{\text{KL}}(\mathcal{N}(Y, \Sigma) \parallel \mathbb{Y}_k^*), \end{aligned}$$

where we identify the surrogate distribution as

$$\mathbb{Y}_k^* \propto (\mathbb{Y}^*)^{\frac{\eta}{\eta+1}} (q_{Y_k})^{\frac{1}{\eta+1}}.$$

Substituting the explicit form of $\mathbb{Y}^*$ completes the proof.

B. Implementation Details

a) *Elite Sample Selection:* To improve convergence and reduce variance, we incorporate elite-set selection within the importance sampling step. Given M rollouts, we define an elite subset $\mathcal{M}_e \subset \{1, \dots, M\}$ containing samples in the lowest K_{elite} -th percentile of cost. Importance weights for $m \notin \mathcal{M}_e$ are set to zero, while for $m \in \mathcal{M}_e$:

$$w_m = \frac{\exp(-\gamma J(Y_m)/\tau)}{\sum_{j \in \mathcal{M}_e} \exp(-\gamma J(Y_j)/\tau)}, \quad (30)$$

where $J(\cdot)$ is the trajectory cost and τ is an adaptive temperature. This selective pressure ensures updates are driven by successful explorations, enabling efficient navigation of non-convex cost landscapes.

b) *Momentum-Accelerated Exponential Moving Average:* To stabilize convergence and accelerate optimization, we apply a momentum-based update scheme. Let $\hat{Y}_{k+1}$ denote the candidate trajectory computed as the weighted expectation over the elite set $\hat{Y}_{k+1} = \sum_{m \in \mathcal{M}_e} w_m (Y_k + \varepsilon_m)$. We compute the update direction $\Delta_k = \hat{Y}_{k+1} - Y_k$ and maintain a momentum buffer v_k via exponential moving average:

$$v_{k+1} = \beta v_k + (1 - \beta) \Delta_k,$$

where $\beta \in [0, 1)$ is the momentum decay coefficient. The trajectory is then updated as: $Y_{k+1} = Y_k + \lambda v_{k+1}$, where $\lambda \in (0, 1]$ is the step size. This two-stage temporal regularization dampens oscillations through directional smoothing while providing momentum for navigating non-smooth cost regions. We also allow the Adam-type [9] gradient update rule as an option.

REFERENCES

- [1] Constantinos Chamzas, Carlos Quintero-Pena, Zachary Kingston, Andreas Orthey, Daniel Rakita, Michael Gleicher, Marc Toussaint, and Lydia E Kavraki. Motion-benchmark: A tool to generate and benchmark motion planning datasets. *IEEE Robotics and Automation Letters*, 7(2):882–889, 2021.

- [2] Zinuo Chang, Hongzhe Yu, Patricio Vela, and Yongxin Chen. Efficient iterative proximal variational inference motion planning. *Robotics and Autonomous Systems*, page 105267, 2025.
- [3] Yongxin Chen, Tryphon T Georgiou, and Michele Pavon. On the relation between optimal transport and schrödinger bridges: A stochastic control viewpoint. *Journal of Optimization Theory and Applications*, 169(2):671–691, 2016.
- [4] Yongxin Chen, Sinho Chewi, Adil Salim, and Andre Wibisono. Improved analysis for a proximal algorithm for sampling. In *Conference on Learning Theory*, pages 2984–3014. PMLR, 2022.
- [5] Lucas C Cosier, Rares Iordan, Sichelukwanda NT Zwane, Giovanni Franzese, James T Wilson, Marc Deisenroth, Alexander Terenin, and Yasemin Bekiroglu. A unifying variational framework for gaussian process motion planning. In *International Conference on Artificial Intelligence and Statistics*, pages 1315–1323. PMLR, 2024.
- [6] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal. Stomp: Stochastic trajectory optimization for motion planning. In *2011 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4569–4574. IEEE, 2011.
- [7] Hilbert J Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of statistical mechanics: theory and experiment*, 2005(11):P11011, 2005.
- [8] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, 30(7):846–894, 2011.
- [9] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [10] Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018.
- [11] Mustafa Mukadam, Ching-An Dong, Xinyan Yan, and Byron Boots. Gaussian process motion planning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5216–5223. IEEE, 2017.
- [12] Mustafa Mukadam, Ching-An Dong, Xinyan Yan, and Byron Boots. Gaussian process motion planning for multi-robot systems. *The International Journal of Robotics Research*, 37(11):1374–1394, 2018.
- [13] Neal Parikh and Stephen Boyd. Proximal algorithms. *Foundations and Trends in Optimization*, 1(3):127–239, 2014.
- [14] Luka Petrović, Juraj Peršić, Marija Seder, and Ivan Marković. Stochastic optimization for trajectory planning with heteroscedastic gaussian processes. In *2019 European Conference on Mobile Robots (ECMR)*, pages 1–6. IEEE, 2019.
- [15] Luka Petrović, Ivan Marković, and Ivan Petrović. Mixtures of gaussian processes for robot motion planning using stochastic trajectory optimization. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 52(12):7378–7390, 2022.
- [16] Mike Phillips and Maxim Likhachev. Asymptotically optimal motion planning using incremental sampling-based algorithms. In *Robotics: Science and Systems (RSS)*, 2012.
- [17] Thomas Power and Dmitry Berenson. Constrained stein variational trajectory optimization. *IEEE Transactions on Robotics*, 2024.
- [18] Nathan Ratliff, Matt Zucker, J Andrew Bagnell, and Siddhartha Srinivasa. Chomp: Gradient optimization techniques for efficient motion planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 489–494, 2009.
- [19] Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999.
- [20] John Schulman, Jonathan Ho, Alex Lee, Ilya Awwal, Henry Bradlow, and Pieter Abbeel. Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research*, 33(9):1251–1270, 2014.
- [21] Evangelos Theodorou, Jonas Buchli, and Stefan Schaal. A generalized path integral control approach to reinforcement learning. 11(Nov):3137–3181, 2010.
- [22] Emanuel Todorov. Efficient computation of optimal actions. *Proceedings of the National Academy of Sciences*, 106(28):11478–11483, 2009.
- [23] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033. IEEE, 2012.
- [24] Marc Toussaint. Robot trajectory optimization using approximate inference. In *International Conference on Machine Learning*, pages 1049–1056, 2009.
- [25] Grady Williams, Paul Drews, Brian Goldfain, James M Rehg, and Evangelos A Theodorou. Model predictive path integral control: From theory to parallel computation. 40(2):344–357, 2017.
- [26] Grady Williams, Paul Drews, Brian Goldfain, James M Rehg, and Evangelos A Theodorou. Information-theoretic model predictive control: Theory and applications to autonomous driving. 34(6):1603–1622, 2018.
- [27] Hongzhe Yu and Yongxin Chen. A gaussian variational inference approach to motion planning. *IEEE Robotics and Automation Letters*, 8(5):2518–2525, 2023.
- [28] Matt Zucker, Nathan Ratliff, Anca Dragan, Michael Pivtoraiko, Matthew Klingensmith, Chris Dellin, J Andrew Bagnell, and Siddhartha Srinivasa. Chomp: Covariant hamiltonian optimization for motion planning. *The International Journal of Robotics Research*, 32(9–10):1164–1193, 2013.